

PROSAFE

Smart Helmet System

Viva Questions and Model Answers

A practical defense guide covering the implemented IoT device, backend pipeline, machine-learning service, user monitoring interface, database, security, testing, limitations, and future work.

Prepared from the current project repository

September 2026

How to use this guide

Practice each answer aloud in your own words. Start with the direct answer, then add one implementation detail and one reason. Do not memorize every sentence. Examiners usually prefer clear understanding over rehearsed wording.

1	Answer directly State the purpose or decision in one sentence.
2	Give evidence Mention the relevant component, endpoint, model, field, or algorithm.
3	Explain the reason Connect the decision to safety, reliability, maintainability, or usability.

Contents

1. Project overview and motivation	Questions 1-6
2. Architecture and end-to-end flow	Questions 7-12
3. Helmet hardware and firmware	Questions 13-18
4. Machine learning	Questions 19-30
5. Prediction reliability and emergency design	Questions 31-36
6. Frontend and user management	Questions 37-46
7. Backend, data, security, and testing	Questions 47-55
8. Critical evaluation and future work	Questions 56-63

Important defense point: the current model result is strong on synthetic data, but it is not evidence of medical validity or guaranteed field performance. State this limitation confidently.

1. Project overview and motivation

Be ready to explain the problem, objectives, users, and value of the system.

Q1. What is the ProSafe Smart Helmet System?

Model answer: ProSafe is an IoT and machine-learning safety-monitoring system for construction workers. An ESP32-based helmet measures physiological and environmental conditions, a backend processes the readings, an ML service estimates risk as SAFE, WARNING, or CRITICAL, and a web application presents monitoring, alerts, analytics, and user-management functions.

Q2. What problem does this project solve?

Model answer: Construction workers can be exposed to heat, harmful gas, high noise, UV radiation, physiological stress, and emergencies. Traditional supervision may not continuously observe every worker. ProSafe provides continuous, worker-linked measurements and early risk indications so supervisors can respond more quickly.

Q3. Who are the main users of the system?

Model answer: The two account roles are ADMIN and WORKER. Administrators manage users and helmets and monitor organization-wide safety information. Workers can view their own profile, readings, predictions, notifications, and guidance, but cannot view another worker's protected data.

Q4. What is the main innovation in the project?

Model answer: The strongest design idea is personalized physiological monitoring. The system uses each worker's baseline heart rate and body temperature and computes percentage deviations, rather than interpreting every person only through identical fixed values. It combines this personalization with environmental sensing and an operational alert workflow.

Q5. What are the main components?

Model answer: There are four main components: ESP32 helmet firmware and sensors; an Express and MongoDB backend; a Python ML service exposed through Flask; and a React/Vite frontend. They communicate through HTTP APIs and persisted database state.

Q6. Why is this project relevant to construction safety?

Model answer: Construction sites change continuously and risks can be local to one worker. A wearable device can follow the worker, link exposure to identity and location, and provide evidence over time. The system supports earlier intervention and gives supervisors a centralized view instead of relying only on occasional manual checks.

2. Architecture and end-to-end flow

Explain the complete journey of one packet and the responsibilities of each layer.

Q7. Describe the end-to-end flow of a normal sensor packet.

Model answer: The helmet reads its sensors and sends a packet to POST /api/helmet/data about every 60 seconds. The backend validates it, resolves the worker from the helmet assignment, obtains baselines, calculates deviations and exposure durations, creates the ML feature vector, calls Flask, filters by confidence, smooths accepted predictions, detects a state transition, generates alerts and notifications when required, stores the packet, and updates the helmet risk command.

Q8. Why did you separate the system into multiple services?

Model answer: Each service owns a clear concern. Firmware owns sensing and physical feedback, Node owns application rules and persistence, Python owns model inference, and React owns presentation. This reduces coupling, allows the model to be replaced without rewriting the web backend, and lets each technology be used for what it does well.

Q9. Why use HTTP between Node and Flask?

Model answer: The trained artifacts are Python joblib/pickle objects that are naturally loaded by scikit-learn and XGBoost. An HTTP boundary lets JavaScript request a prediction using JSON without starting Python for every packet or attempting to interpret Python artifacts directly.

Q10. What does the Flask wrapper do?

Model answer: It loads the saved model, scaler, label encoder, and metadata once at startup. It exposes GET /health and POST /predict, validates the ten numeric features, calls SafetyPredictor, normalizes labels to uppercase, and returns the predicted class with probabilities.

Q11. Why is business logic kept in Node instead of Flask?

Model answer: The model should answer only the prediction question. Node controls identity, baselines, validation, confidence thresholds, smoothing, state transitions, alerts, notifications, database records, emergency state, and commands. This keeps ML replaceable and prevents model code from becoming the entire safety application.

Q12. What happens if the ML service is unavailable?

Model answer: The backend does not crash and does not invent a SAFE prediction. It records why ML was skipped, persists the raw packet, preserves the worker's last accepted risk state, and continues sending that last known state to the helmet. This is graceful degradation.

3. Helmet hardware and firmware

Connect each physical component to the data it contributes and explain device-side resilience.

Q13. Why was the ESP32 selected?

Model answer: The ESP32 provides sufficient processing power, Wi-Fi connectivity, analog and digital interfaces, I2C, UART, I2S, timers, and FreeRTOS support at low cost. These capabilities suit a prototype that must read several sensors and communicate with a web backend.

Q14. Which sensors are used and what do they measure?

Model answer: The MAX3010x sensor measures heart-rate-related optical signals; LM35 measures body or contact temperature; DHT11 measures ambient temperature and humidity; MQ2 measures combustible gas or smoke response; the I2S microphone estimates noise; a UV sensor estimates UV index; and a GPS module supplies location.

Q15. How frequently does the helmet communicate?

Model answer: Normal packets are sent approximately every 60 seconds. The helmet polls the normal risk-command endpoint every 5 seconds. While emergency is active, it polls the dedicated reset endpoint roughly every 3 seconds. Wi-Fi reconnect attempts and HTTP timeouts are also bounded.

Q16. What does the RGB LED represent?

Model answer: It represents the latest accepted backend risk state: SAFE, WARNING, or CRITICAL. The helmet retains the last valid command when a poll fails instead of assuming SAFE. Emergency mode overrides the ordinary risk display.

Q17. Does the helmet itself run the ML model?

Model answer: No. It collects and validates physical readings, sends packets, polls for commands, and handles immediate emergency behavior. The trained model runs in the Python service, while the backend controls how a prediction changes operational state.

Q18. How does firmware handle unreliable hardware or communication?

Model answer: The firmware includes sensor-validity flags, filtering, DHT retries and fallback behavior, Wi-Fi reconnection, HTTP timeouts, a watchdog, retained backend risk state, and retryable emergency acknowledgement. These mechanisms reduce total failure when one reading or network request fails.

4. Machine learning

Focus on the target, features, personalization, training, evaluation, and limitations.

Q19. What exactly does the ML model predict?

Model answer: It performs multi-class classification and predicts one of three safety-risk labels: Safe, Warning, or Critical. It also provides class probabilities, allowing the backend to evaluate prediction confidence.

Q20. What are the ten input features?

Model answer: They are ambient temperature, UV index, gas PPM, noise dB, body temperature, heart rate, baseline heart rate, baseline body temperature, heart-rate deviation percentage, and body-temperature deviation percentage. Worker ID, timestamp, and GPS are intentionally excluded as predictive features.

Q21. How are personalized deviations calculated?

Model answer: Deviation percentage is calculated as (current value - baseline value) divided by baseline value, multiplied by 100. A heart rate of 105 BPM with a 70 BPM baseline produces a +50 percent deviation.

Q22. Why include both raw values and deviations?

Model answer: The raw values provide absolute context, while deviations show how unusual the reading is for that individual. Together they allow the model to learn patterns such as a moderately high absolute value that is nevertheless a very large personal change.

Q23. Which algorithms were compared?

Model answer: The project compares Random Forest, XGBoost, RBF SVM, and Logistic Regression. Tree models use unscaled inputs, while SVM and Logistic Regression use StandardScaler because their optimization and distance calculations are sensitive to feature magnitude.

Q24. Which model won and how well did it perform?

Model answer: In the saved comparison results, XGBoost achieved approximately 98.88 percent test accuracy and weighted F1. Its five-fold cross-validation weighted F1 mean was approximately 98.76 percent with a standard deviation near 0.15 percentage points.

Q25. Why use weighted F1 in addition to accuracy?

Model answer: Accuracy counts total correct predictions but may hide weak performance on a smaller class. Weighted F1 combines precision and recall for each class while weighting by support, so it is more informative when class frequencies differ.

Q26. Why use a stratified 80/20 split?

Model answer: Eighty percent is used for training and twenty percent for held-out testing. Stratification preserves approximately the same Safe, Warning, and Critical proportions in both sets, reducing evaluation distortion caused by an unbalanced split.

Q27. What is five-fold cross-validation used for?

Model answer: The data is divided into five folds. The model trains on four and validates on the remaining fold, repeating five times. Similar scores across folds suggest that the holdout score is not just the result of a lucky single split.

Q28. What is the purpose of the label encoder and scaler?

Model answer: The label encoder converts text classes into integer targets for training and converts predicted integers back to human-readable labels. The scaler standardizes feature magnitudes for SVM and Logistic Regression. Metadata records whether the selected model requires scaling.

Q29. What is the biggest ML limitation?

Model answer: The reported performance is based on a 20,000-row synthetic dataset covering 90 synthetic workers. High performance shows that the model learned those generated patterns, but it does not prove equivalent clinical or field performance. Real-world data collection, external validation, calibration, and bias analysis are required.

Q30. Could this model diagnose a medical condition?

Model answer: No. It is a project risk classifier, not a medical diagnostic system. Sensor limitations, synthetic training data, baseline quality, environmental context, and unvalidated thresholds prevent medical claims. It should support human safety decisions rather than replace trained personnel.

5. Prediction reliability and emergency design

Be precise about confidence, smoothing, transitions, and the separation of emergency state.

Q31. What is confidence filtering?

Model answer: The backend accepts a prediction only when the probability of its predicted class meets the configured threshold, which defaults to 0.70. A lower-confidence result is stored but does not update prediction history or operational risk state.

Q32. Why smooth predictions?

Model answer: Wearable sensors can fluctuate because of movement, contact quality, or brief environmental changes. The backend performs a majority vote over the latest accepted predictions, with a default window of five, so one noisy reading is less likely to trigger a state change.

Q33. How are ties handled during majority voting?

Model answer: A tie is resolved toward the more severe state. For a safety system, the implementation prefers a possible false alarm over silently selecting a lower-risk state when the evidence is evenly divided.

Q34. When is an alert created?

Model answer: A normal alert is created only when the smoothed state genuinely changes, such as SAFE to WARNING or WARNING to CRITICAL. Repeated predictions that keep the same state do not generate duplicate transition alerts.

Q35. Why is EMERGENCY not a fourth ML class?

Model answer: Emergency represents an explicit human action, not a statistical inference. A button press bypasses baselines, features, confidence filtering, and ML. Keeping emergency state separate ensures a SAFE model prediction can never cancel a worker's request for immediate help.

Q36. How is an emergency reset completed?

Model answer: An administrator requests a reset, the helmet polls the dedicated reset endpoint, clears its local emergency state, and sends an acknowledgement. Only that acknowledgement finalizes the backend state, making the workflow resilient to a missed network response.

6. Frontend and user management

Explain why the user page is an operational safety view rather than ordinary CRUD.

Q37. What does the user-management page provide?

Model answer: Administrators can search, filter, add, view, edit, and soft-delete users. Worker records also connect an identity to a helmet, live and last-known readings, location, prediction state, emergency status, sensor histories, and recommended actions.

Q38. Why is helmet assignment important?

Model answer: Normal firmware packets identify the helmet rather than the worker. User.helmetId is the authoritative assignment used to resolve the worker. This creates identity-to-device traceability and prevents a packet from being attributed to an unrelated account.

Q39. What is special about the View User page?

Model answer: For a worker it becomes a compact monitoring workspace. It shows identity, assigned helmet, latest six sensors, online state, last update, location, current condition, guidance, and a clickable risk badge that opens prediction history. Admin records only show administrative information.

Q40. What do the personalized sensor modals show?

Model answer: Heart rate and body temperature modals show the current value, the worker's baseline, percentage deviation, timestamp, and seven-day daily averages. They avoid fixed population risk bands because those signals are interpreted relative to the individual.

Q41. What do environmental sensor modals show?

Model answer: Noise, gas, UV, and ambient-temperature modals show the current value, backend-classified category, configured Safe/Warning/Critical ranges, a reference label, and seven-day averages. The frontend does not duplicate threshold numbers.

Q42. What appears in the Safety Prediction modal?

Model answer: It shows the current smoothed risk state, the latest accepted prediction with confidence and time, today's transition-compressed prediction timeline, and a separate emergency banner when applicable.

Q43. Why use a categorical timeline rather than a line chart?

Model answer: SAFE, WARNING, and CRITICAL are categories rather than continuous quantities. Colored time segments communicate duration and transitions without suggesting that the labels are numeric measurements.

Q44. What is the Current Condition and Guidance card?

Model answer: It is a backend-driven summary refreshed every 60 seconds. It combines emergency state, ML risk, environmental classifications, personal deviations, and helmet connectivity, then presents the most relevant contributing factors and deterministic recommended actions.

6. Frontend and user management - continued

Q45. How does the UI avoid presenting stale readings as live?

Model answer: When the backend reports the helmet offline, the interface labels measurements as Last known readings and shows the last update time. Separate empty states cover no helmet, no packets, missing history, and authorization failures.

Q46. What frontend improvement is especially important?

Model answer: The current Add/Edit User form does not expose baseline heart rate and baseline body temperature even though ML requires them. A supervised baseline-enrollment workflow with validation, sampling, provenance, and recalibration would close this operational gap.

7. Backend, data, security, and testing

Show that you understand persistence, access control, integrity, and verification.

Q47. Why use MongoDB and Mongoose?

Model answer: Sensor packets and nested prediction results naturally fit document storage, while Mongoose provides schemas, validation, indexes, and a consistent Node API. Collections preserve users, helmets, packets, worker state, commands, alerts, and notifications.

Q48. Why store raw and processed data separately?

Model answer: HelmetData preserves the original sensor values under raw and stores deviations, exposure, and prediction output under processed and prediction. This supports auditing and reprocessing because derived values never overwrite the received evidence.

Q49. Why is WorkerProcessingState stored in the database?

Model answer: Prediction windows, current risk, exposure accumulators, emergency status, and reset state must survive individual requests and backend restarts. Keeping them only in controller variables would lose safety state whenever the process restarted.

Q50. What database indexes support performance?

Model answer: HelmetData is indexed by worker and timestamp, helmet and timestamp, and timestamp alone. Alert has worker/timestamp and operational indexes. These patterns support latest-reading lookup, seven-day aggregation, admin analytics, and recent-alert queries.

Q51. How is authentication implemented?

Model answer: The backend uses JWT authentication and bcrypt password hashing. Protected user routes validate the token, and middleware enforces ADMIN or self-or-admin authorization according to the operation.

Q52. How is access to worker sensor data controlled?

Model answer: An administrator may view any active worker, while a worker may access only the route containing their own user ID. Sensor history and guidance reject unknown, inactive, or non-worker target accounts.

Q53. What prevents a worker/helmet mismatch?

Model answer: If a packet supplies a worker ID, the backend checks whether that worker is actually assigned to the provided helmet. If no worker ID is supplied, it resolves the assigned worker from helmetId. A mismatch is rejected instead of silently storing data under the wrong identity.

Q54. What testing tools are used?

Model answer: The backend uses Jest, Supertest, and mongodb-memory-server for service and API tests. The frontend uses Vitest, Testing Library, jest-dom, user-event, and jsdom. The repository contains tests for authentication, users, sensors, alerts, dashboards, analytics, reports, guidance, settings, roles, and UI components.

Q55. What should be tested before real deployment?

Model answer: Add hardware calibration tests, long-running packet and reconnection tests, ML drift and calibration evaluation, load testing, network security testing, authorization abuse cases, emergency drills, usability studies, failure injection, and validation using representative real workers and site conditions.

8. Critical evaluation and future work

Strong viva answers acknowledge tradeoffs and propose specific next steps.

Q56. What are the main current limitations?

Model answer: The dataset is synthetic; baseline enrollment is incomplete in the UI; several ranges are project-configured placeholders; prototype sensors have accuracy and contact limitations; firmware contains deployment-specific network configuration; unauthenticated device endpoints need production hardening; and a single Flask development server is not a production deployment.

Q57. How would you secure helmet-to-server communication in production?

Model answer: Use HTTPS/TLS, per-device credentials or certificates, signed requests, nonce or timestamp replay protection, rate limiting, secret provisioning outside source code, network segmentation, key rotation, and device revocation. Validate timestamps and retain audit logs.

Q58. How would you improve ML deployment?

Model answer: Serve the app behind a production WSGI server, containerize it, add structured logging and metrics, version models and schemas, validate input ranges, use readiness and liveness probes, monitor latency and drift, and provide a controlled rollback to a previous artifact.

Q59. How would you improve the model scientifically?

Model answer: Collect consented real-world data, define labels with safety experts, split by worker or site to prevent identity leakage, compare calibration and per-class recall, tune thresholds based on cost, evaluate fairness, test on unseen sites, and retrain only through a documented versioned pipeline.

Q60. Why might a random row split overestimate performance?

Model answer: If measurements from the same synthetic worker appear in both train and test sets, the model may benefit from worker-specific patterns that would not transfer to a new worker. A group split by worker and an external site-based test set would provide stronger evidence of generalization.

Q61. How would the system scale to many helmets?

Model answer: Move packet ingestion to a message broker, process asynchronously, horizontally scale stateless API and ML workers, batch or cache appropriate queries, enforce device rate limits, partition high-volume telemetry, use time-series retention policies, and monitor queues and service latency.

Q62. What is the strongest defense of your design?

Model answer: The project does not let one raw ML output control the whole safety process. It preserves raw evidence, personalizes physiological signals, checks confidence, smooths predictions, separates explicit emergencies, persists state, exposes last-known status, and keeps deterministic application rules around the model.

Q63. If you had one more development phase, what would you prioritize?

Model answer: First implement supervised baseline enrollment and real-world validation because model quality depends on trustworthy baselines and representative data. In parallel, secure device authentication and TLS, then run an end-to-end field pilot with calibrated sensors and documented safety oversight.

Last-minute revision sheet

Use these facts to anchor longer answers.

Risk classes	SAFE, WARNING, CRITICAL
Emergency	Separate manual state; never an ML class
ML inputs	10 features: 6 live, 2 baselines, 2 deviations
Best saved model	XGBoost, about 98.88% weighted F1 on synthetic test data
Dataset	20,000 synthetic rows; 90 synthetic workers
Validation	Stratified 80/20 split and five-fold stratified cross-validation
Confidence gate	Default 0.70
Smoothing	Majority vote over latest 5 accepted predictions; severe state wins ties
Packet period	About 60 seconds
Command poll	About 5 seconds
ML endpoint	Flask POST /predict; model loaded once at startup
Core stack	ESP32/Arduino, Express/Node, MongoDB/Mongoose, Flask/Python, React/Vite
Audit design	Raw readings preserved separately from derived values and predictions
Key fail-safe	Do not fabricate SAFE when ML, baseline, or network is unavailable
Priority improvement	Baseline enrollment plus real-world validation and device security

A strong closing statement

ProSafe is a prototype decision-support system, not a replacement for safety professionals or medical diagnosis. Its engineering value comes from combining wearable sensing, personalized features, cautious prediction handling, persistent audit data, explicit emergency control, and an interface that turns technical signals into actionable context.